



VoicePls — AI Solutions Agency

SOFTWARE DESIGN DOCUMENT

Digital Restaurant Menu & Service Platform



Client: Nobby Restaurant & Lounge

Client Name	Taha Khan
Prepared By	Syed Owais Ali Shah
Project Manager	Syed Owais Ali Shah & Aqib Ali Ghumro
Project Lead	Abdul Samad Siddiqui
Agency	VoicePls (voicepls.com)
Date	05 August 2026

1. Purpose & Scope

This Software Design Document (SDD) defines the technical architecture, technology stack, database design, and data flow for the Nobby Restaurant & Lounge digital menu and table-service platform. It translates the approved functional requirements into an engineering blueprint for the development team.

2. Recommended Technology Stack

2.1 Frontend

Recommendation: Next.js (React) with Tailwind CSS.

- Server-side rendering + static generation for fast menu load times on customer phones.
- Excellent SEO and Core Web Vitals — important for a public-facing menu link.
- Large ecosystem, easy integration with animation libraries (Framer Motion) to achieve the cinematic/video-led style referenced in the design section.

2.2 Backend

Recommendation: NestJS (Node.js, TypeScript).

Option	Verdict	Reason
NestJS	Selected	Same language (TS) as frontend, modular architecture, built-in WebSocket support for real-time order/service alerts, strong for team-based long-term maintenance.
FastAPI	Alternative	Great for Python/AI-heavy workloads, but adds a second language to maintain alongside a Next.js frontend.
Django	Not selected	Best for content-heavy monoliths; heavier than needed for a real-time, API-first ordering platform.

2.3 Database

Recommendation: PostgreSQL as the primary database, with Redis for real-time/session data.

Per the CAP theorem, this platform prioritizes Consistency and Partition tolerance (CP) over Availability, because orders, prices, and table sessions must never be inconsistent or lost. PostgreSQL (relational, ACID-compliant) fits this need better than a NoSQL/AP-oriented store like MongoDB.

Option	Verdict	Reason
PostgreSQL	Selected (primary)	ACID transactions for orders/pricing, strong relational structure (tables, menu items, categories, orders), JSONB support for flexible menu attributes.
MongoDB	Not selected	Better for unstructured, high-write, eventually-consistent data — not ideal where order accuracy and consistency matter most.
MySQL	Alternative	Viable relational alternative to Postgres; Postgres chosen for richer JSON and indexing features.

Redis	Selected (cache/real-time)	Powers live request dashboard, table sessions, and pub/sub for instant staff notifications.
-------	----------------------------	---

2.4 Real-Time Communication

WebSockets (Socket.IO via NestJS Gateway) for instant order and service-call notifications to the staff dashboard, and Redis pub/sub to scale WebSocket delivery across multiple server instances.

2.5 Hosting & Infrastructure

- Frontend: Vercel (Next.js native hosting)
- Backend: Dockerized NestJS on AWS/DigitalOcean
- Database: Managed PostgreSQL (AWS RDS) + Managed Redis
- Media/Images: Cloud storage (AWS S3 / Cloudinary) with CDN delivery
- Scheduled Job: weekly cron (Sunday reset per requirements) to archive/clear old order data

3. System Architecture Overview

The system follows a client-server architecture with three main layers:

- Presentation Layer: Next.js customer menu (mobile-first) + Next.js staff/admin dashboard
- Application Layer: NestJS REST + WebSocket API handling orders, menu management, and service calls
- Data Layer: PostgreSQL (persistent data) + Redis (live sessions, cache, pub/sub)

4. Data Flow Summary

- Customer scans table QR / enters table number → session created (Redis) → menu loads from PostgreSQL via API.
- Customer places order or presses a service button → NestJS API writes to PostgreSQL → event pushed via WebSocket to Live Request Dashboard.
- Staff updates menu (items/prices/photos/banners) → written to PostgreSQL → cached in Redis → reflected instantly on customer menu.
- Weekly scheduled job clears completed order data every week to keep the system fast.

5. Frontend Design Direction

The customer-facing menu should follow a premium, cinematic restaurant presentation style, combining full-screen video/imagery, elegant typography, and smooth scroll animations, based on the following references:

- Hero video/landing style: <https://gaststatte.framer.website/>
- Menu layout & item presentation style: <https://qitchen-template.framer.website/>
- Overall visual theme & mood: <https://terrona.framer.website/>

Implementation notes: Next.js + Tailwind CSS + Framer Motion for animations; optimized video/image loading for fast mobile performance; dark, upscale color palette consistent with the Nobby brand (charcoal grey + red accent, per the Nobby logo).

6. Document Control

Field	Detail
Prepared By	Syed Owais Ali Shah
Project Manager	Syed Owais Ali Shah, Aqib Ali Ghumro
Project Lead	Abdul Samad Siddiqui
Client	Taha Khan — Nobby Restaurant & Lounge
Agency	VoicePls